

AORCEA – An adaptive operator rate controlled evolutionary algorithm

M. Giger *, D. Keller, P. Ermanni

Centre of Structure Technologies, ETH Zurich, Leonhardstr. 27, LEO C4, CH-8092 Zurich, Switzerland

Received 20 December 2006; accepted 20 December 2006

Available online 14 February 2007

Abstract

When applying evolutionary algorithms to optimization problems many different strategy parameters have to be set to define the behavior of the evolutionary algorithm itself. To a certain extent these strategy parameter values determine whether the algorithm is capable of finding a near-optimum solution or not. In particular the choice of the different genetic operators and their relative rates is most often based on experience. Furthermore, the operator rates are defined before starting the optimization runs and remain unchanged until the stopping criterion is reached. Controlling the parameter values during the run has the potential of adjusting the algorithm to the problem while solving the problem. This paper investigates an adaptive strategy controlling the rates of arbitrary chosen genetic operators. The control mechanism is based on the state of the optimization by evaluating a success and a diversity measure for each operator. More efficient operators are favored in order to find better solutions with less evaluations. The algorithm is tested with constrained and unconstrained numerical examples and a concrete structural optimization problem is treated.

© 2007 Elsevier Ltd. All rights reserved.

Keywords: Evolutionary algorithm; Strategy parameter control; Adaptive operator rates; Structural optimization

1. Introduction

During the last two decades much of research has been invested in the field of adaptive evolutionary algorithms. The motivation to investigate a wide variety of adaptation techniques is the very appealing idea to adjust the evolutionary algorithm (EA) to the problem while solving the problem itself.

Each EA consists of components which need to be defined. These components are genetic operators such as mutation and crossover, selection mechanisms for selecting parents for reproduction, replacement strategies determining which individuals are kept in the current population, etc. Each of these components is defined by strategy parameters, e.g. population size, operator rates, or tournament size of selection, which have to be set. The choice of

good parameter values is most often based on experience and requires therefore considerable effort. De Jong [1] experimentally developed a heuristic for a traditional GA with one-point crossover and bit mutation which proved to be efficient for a number of numerical problems. Later studies using a meta-algorithm [2] to find suitable values or using exhaustive testing [3] led to further recommendations regarding the parameter values. Despite these recommendations there is no evidence that there exists a parameter setup which is optimal for an entire optimization run. Consequently, adapting the parameter values on the run is a promising way to improve EA performance.

A quite popular classification which distinguishes three levels of adaptation, i.e. the population-level, the individual-level, and the component-level¹ is presented by Angeline [4].

* Corresponding author. Tel.: +41 44 632 71 72; fax: +41 44 633 11 25.
E-mail addresses: mgiger@ethz.ch (M. Giger), davkelle@ethz.ch (D. Keller), permanni@ethz.ch (P. Ermanni).

¹ Note that component here describes a subindividual structure, e.g. a single gene, while in this paper the term component is generally used for parts of an evolutionary algorithm, i.e. operators, fitness definition, selection, etc.

- **Population-level:** Adaptive parameters which are global to the entire population and directly influence the distribution of the individuals of the subsequent generation(s) are dynamically adjusted. An early and well-known example for this type of parameter adaptation is the Rechenberg's '1/5 success rule' in (1 + 1)-evolution strategies [5]. Further examples for population-level adaptive techniques are the various meta-algorithms, e.g. Grefenstette [2], Kakazu et al. [6], Freisleben and Hartfelder [7]. Moreover, Julstrom [8], Tuson [9], and Davis [10] define adaptation rules for operator rates based on the operator success or productivity. Algorithms adjusting the population size during the optimization, e.g. Smith [11] and Hinterding et al. [12], fall also into the category of population-level adaptation.
- **Individual-level:** Adaptive techniques associate separate strategy parameters with each individual of the population determining which of an individual's representational components are to be modified. The extension of genotypes with bits coding for the mutation rate of the respective individual is investigated by Bäck [13] for generational genetic algorithms. Further examples for individual-level adaptive techniques can be found in Hinterding et al. [12], Spears [14], Srinivas and Patnaik [15], and Arabas et al. [16].
- **Component-level:** These algorithms also associate strategy parameters with each individual, but in contrast to individual-level adaptations they define how each representational component of the individual is modified. An example for such an algorithm is presented by Bäck [13] where a real-valued mutation rate is added to each gene of the individuals. A further example is Smith's [17] Lego mechanism which is further developed by Smith and Fogarty [18] to the Apes algorithm.

Eiben et al. [19] propose a more comprehensive classification which is partially based on Angeline's [4] classification presented above. In particular, Angeline's update mechanisms (absolute and empirical) are further divided. The classification considers four different aspects:

- (1) *what* is changed: representation, evaluation function, genetic operators, operator rates, selection operator, replacement operator, population size, etc.,
- (2) *how* is it changed: deterministic, adaptive, or self-adaptive update mechanism,
- (3) *on what level* it is changed: population, individual, or component level (according to [4]),
- (4) and based upon which *evidence* the change is made: performance of operators, diversity of population, etc.

Item (2) deserves closer attention since there are three different categories needing some more explanation:

- **Deterministic** parameter control.
No feedback from the search process is used and the strategy parameters are altered deterministically. Nor-

mally, the rule is applied periodically after a given number of generations.

- **Adaptive** parameter control.

This adaptation mechanism is based on feedback from the search process. The magnitude and the direction of change of the strategy parameters directly depend on predefined measures.

- **Self-adaptive** parameter control.

The self-adaptation of strategy parameters is inspired by the idea of "evolution of evolution". The strategy parameters are directly encoded in the representation and are subject to mutation and recombination like ordinary genes.

In this paper an EA providing a robust operator setting for a wide variety of optimization problems is presented. AORCEA – the adaptive operator-rate controlled evolutionary algorithm – can be categorized as follows: AORCEA works on the genetic operator rates in an adaptive way where the changes of the strategy parameters affect the population-level and the evidence of change is based on the success of operators as well as on the diversity of the population.

The next section gives a detailed description of AORCEA followed by some numerical benchmark function optimizations in Section 3. In Section 4 the performance of this novel algorithm is tested by optimizing a tubular steel trellis frame of a motorcycle and Section 5 concludes the paper.

2. AORCEA – adaptive operator rate controlled EA

The motivation for introducing an adaptive strategy arose from the recurrent problem of setting appropriate operator rates for structural optimization problems. The basis of AORCEA is a rather simple EA as presented in Fig. 1.

2.1. The components of AORCEA

The standard components of such an algorithm are extended with an additional component termed *adaptation*.

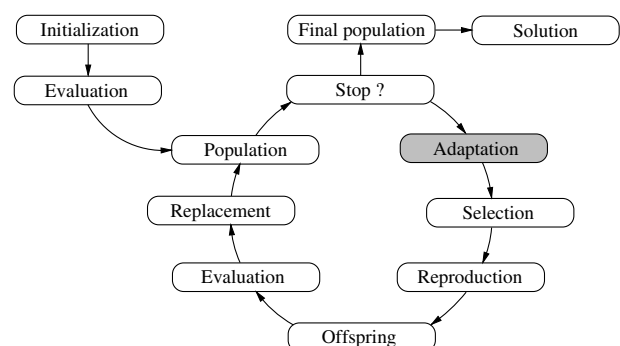


Fig. 1. Basic scheme of an adaptive evolutionary algorithm.

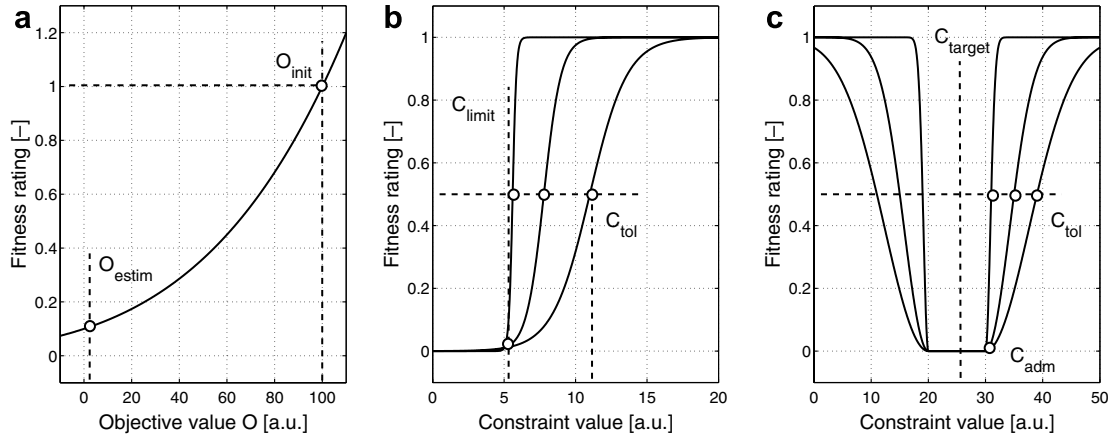


Fig. 2. Fitness functions. The constraint functions (b) and (c) are depicted with variations of the parameter C_{tol} . (a) Design objective, (b) limit constraint and (c) target constraint.

Theoretically, arbitrary control parameters of the evolutionary algorithm can be adjusted in the adaptation phase, e.g. the population size, some operator rates, the fitness definitions, etc. Within the scope of AORCEA only the operator rates of the applied genetic operators are adjusted on the basis of success and diversity measures.

Many journal papers concerned with (self-)adaptive strategies investigate Genetic Algorithms (GA) [20] which are based on bit representation and only employ simple operators like one-point crossover and bit mutation. In contrast, AORCEA is based on a more general evolutionary computing framework (Evolving Objects²) which provides an environment to evolve any kind of data structures. Thus, there are some basic differences between the traditional GA and AORCEA which are presented next.

- **Initialization and Representation.** For structural optimization it is very convenient to use a heterogeneous genotype, e.g. the *eoUniGene* concept presented by König [21], which consists of different gene types. The most common gene types are the *float-gene* and the *integer-gene* representing an arbitrary floating-point or integer parameter, respectively. For both gene types upper as well as lower limits can be provided and also a σ -value needs to be defined for Gaussian mutation of the respective genes. Further gene types provided by the *eoUniGene* concept are: *bool-gene*, *string-gene*, *float-list-gene*, and *const-float-list-gene*, all of them having additional properties which are useful for the representation of structures to be optimized. The initialization of the genotype is completely uniform, i.e., for each gene a random value is determined.
- **Evaluation.** An EA-based optimization requires the definition of a fitness value assessing the quality of each individual. For that purpose the fitness formulations introduced by König [21] are employed defining the fit-

ness value as a weighted sum of ratings for the optimization objective and potential constraints. For the fitness value F of a given individual $\mathbf{i}$ it holds

$$F(\mathbf{i}) = \sum_j w_j D_j(\mathbf{i}), \quad (1)$$

where w_j denotes the respective weights and D_j stands for mapping functions assigning fitness ratings within an interval $[0,1]$ to the given individual. Three different types of mapping functions are distinguished, namely, design objective, limit constraint, and target constraint functions. In Fig. 2 examples are shown for the design objective, the limit constraint, and the target constraint functions. The design objective function is defined by two parameters O_{init} and O_{estim} . The first parameter determines for which objective value a fitness portion of 1 is assigned and the second parameter is an estimation of the objective value which can be reached during the optimization (fitness portion 0.1). A smooth step function is chosen for the limit constraint, whereas the exact limit constraint value C_{limit} is penalized with 0.01 and C_{tol} maps the constraint value to a fitness portion of 0.5. Accordingly, the symmetric target constraint function is defined by three values. C_{target} and C_{tol} correspond to the respective parameters of the limit constraint and C_{adm} additionally defines a tolerance interval for slightly missed target constraint values.

- **Selection.** In the selection phase some individuals are chosen from the current population to become parents of the subsequent generation. AORCEA uses the so-called *stochastic tournament selection* which chooses two random individuals of the population and prefers the better individual with a given probability.
- **Reproduction.** In the reproduction phase the parents are recombined using crossover operators or are simply mutated by applying mutation operators. The class of exchange crossover operators takes two parents and creates two offspring individuals by exchanging gene values of the parents. For example, the one-point crossover, the two-point crossover, the general n -point crossover, the

² <http://eodev.sourceforge.net>.

uniform crossover, and the intermediate crossover belong to this class. The class of arithmetic crossovers creates offspring by computing two linear combinations of the parents for every gene of the two involved parents. Examples of this class are the segment crossover using a random factor $\alpha \in [0, 1]$ for the linear combination for the entire genotype, whereas the hypercube crossover redetermines α for each gene. Furthermore, uniform and Gaussian mutation operators are applied to insert diversity into the population by mutating each gene with a certain probability. Each of these operators is assigned with a unique operator rate determining its probability of application during the evolutionary process. AORCEA chooses a single operator from the set of available operators to produce child individual(s). This style of algorithm which is successfully applied in Tuson [9] and Davis [10] is used to estimate success and diversity measures for each operator application. Then, these measures are taken as a basis for the adaptation of the respective operator rates.

- **Replacement.** AORCEA is based on the *plus replacement* known from Evolution Strategies (ES) where the best from offspring and parents become the members of the next generation.

2.2. Adaptation strategy

Evolutionary search is always an interaction between exploration of new regions and exploitation of previously detected good regions of the search space. The adaptation strategy proposed within this section, schematically shown in Fig. 3, focusses on determining whether exploitative or explorative operators should be favored in the current search state.

First, a best-fitness-frequency measure is evaluated indicating the success of the current search. Depending on this value either exploitative or explorative genetic operators are forced in order to accelerate the search process. In other words, if the search process is too exploitative, i.e. the search stagnates in a local optimum, the operator rates

of explorative genetic operators are increased. On the other hand, if the search process is successful the rates of operators finding above-average offspring are increased in order to explore the region around the current best solution. Consequently, the next step is to determine which operators are of explorative and which are of exploitative nature. For that purpose a *success* and a *diversity* measure are introduced. The success measure indicates which genetic operators are able to produce offspring individuals of above-average quality, whereas the diversity measure determines how exploitative the genetic operators are. Then, the operators are ranked based on these measures and their rates are adapted, whereas the rates of operators with low rank are increased and operators having a high rank get a lower probability of application.

The subsequent sections describe the best fitness frequency measure, the success as well as the diversity measure, and the operator rate adaptation in detail.

2.2.1. The best fitness frequency measure

A popular method to investigate the search state is the analysis of the so-called fitness landscape³ as for example shown by Puisa [22]. Although the exact shape of the fitness landscape is generally unknown for structural optimization problems, the distribution of the fitness values on the fitness landscape indicates the search state. If all the fitness values of the current population are extremely close to each other, this either indicates that the search stagnates in a at least local optimum or that the fitness landscape is very flat. On the other hand, a population containing individuals with very different fitness values indicates a rather steep uni- or multimodal fitness landscape where the search process quite often finds improved solutions.

A simple analysis of the search state can therefore be estimated by calculating the so-called best fitness frequency (bff). The bff value is based on the frequency distribution of the fitness values of the current population. As shown in Fig. 4, a histogram of the fitness values can be established. The class limits defining the class intervals are determined by f_B , i.e. the fitness of the current best individual, and a factor ε determining the width of the class interval. The bff value of a single individual i of the current population is calculated as

$$\text{bff}_i = \begin{cases} 1, & \text{if } |f_i - f_B| \leq f_B \cdot \varepsilon \\ 0, & \text{else,} \end{cases} \quad (2)$$

where f_i is the fitness value of individual i . The total bff value of the current population is then calculated as

$$\text{bff} = \frac{1}{N} \sum_i \text{bff}_i, \quad (3)$$

³ The fitness landscape represents a hypersurface in the $(n + 1)$ -dimensional space, where n is the number of genes in the genotype and the additional dimension stands for the fitness value. Each individual can therefore be represented as a point on this hypersurface.

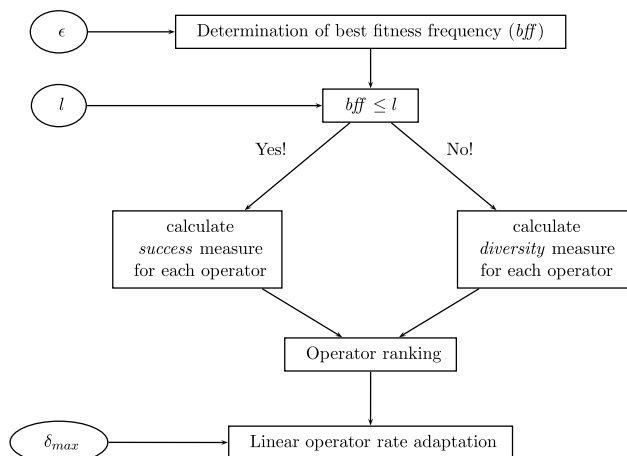


Fig. 3. Basic scheme of an adaptive evolutionary algorithm.

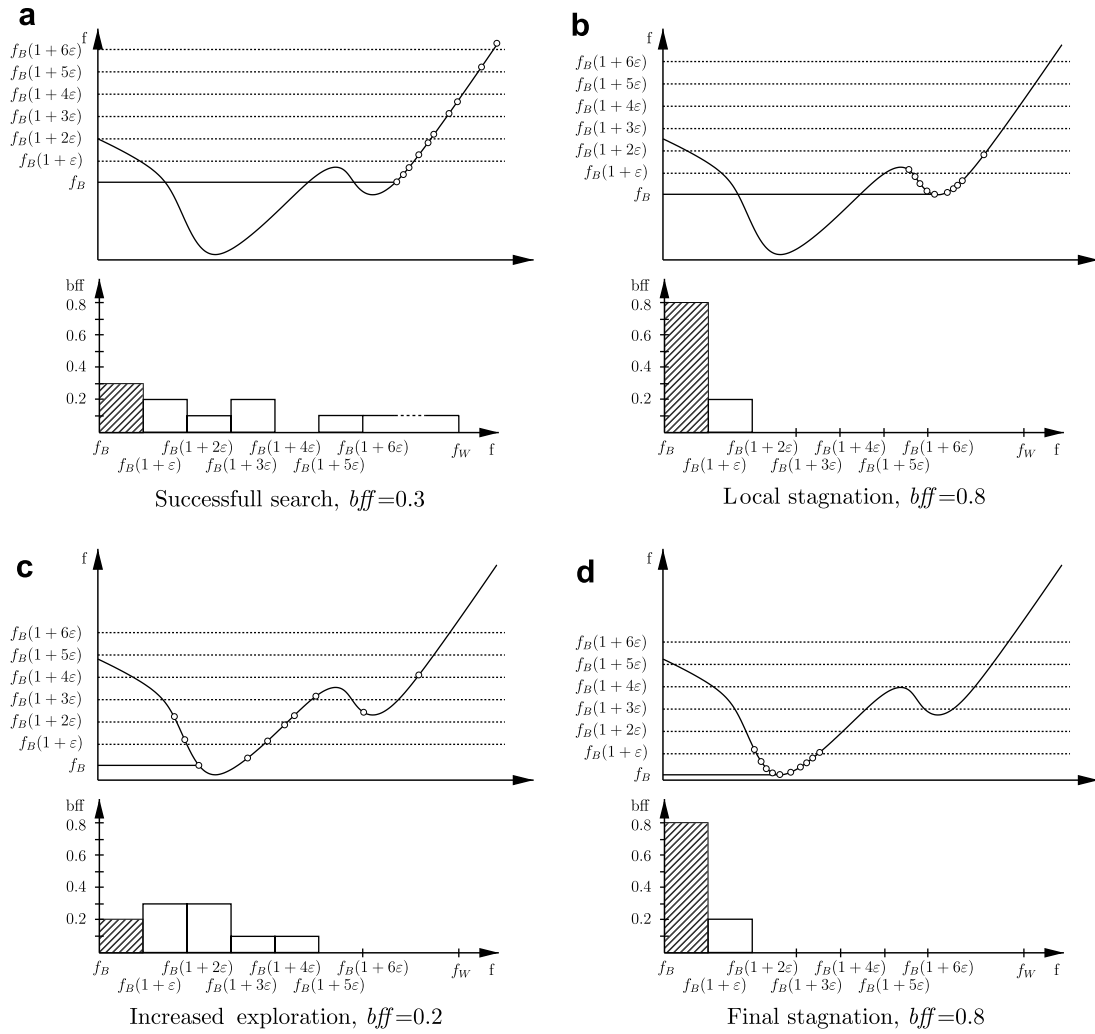


Fig. 4. Schematic illustration of different search states of an evolutionary search on the fitness landscape. Shaded areas of the histograms indicate the bff value (relative frequency of the most left class interval).

where N denotes the population size. In other words, the bff value corresponds to the relative frequency of the interval containing the current best fitness value.

Fig. 4 schematically illustrates different states of the search process with the corresponding bff values. A rather low bff value can be found in Fig. 4 where the search is successful on a rather steep region of the fitness landscape. Only few individuals are very close to the current best solution since in almost every new generation an improved individual is found. In Fig. 4 the search reaches a local optimum and only slightly improved individuals can be found, hence, the bff value is increased. If the search algorithm is able to escape from this local optimum, the bff value is reduced again as illustrated in Fig. 4. Finally, the search algorithm reaches another at least local optimum in Fig. 4 and the bff value is increased again.

Summarizing, low bff values indicate a rather successful or very explorative search. On the other hand, high bff values indicate stagnation in a at least local optimum. By introducing a threshold value $l \in [0, 1]$ defining a balance

between exploitation and exploration, the adaptation strategy can be described as follows:

- $\text{bff} \leq l$: Increase the operator rates of successful operators contributing above average to the search process performance.
- $\text{bff} > l$: Increase the operator rates of explorative operators in order to escape from the local optimum by introducing more diversity into the population.

Thus, measures for success and diversity of the applied operators need to be defined.

2.2.2. Operator success measure

In general, the evolutionary search process should always be a minimization problem. Thus, the success measure for the application of an operator of type X in generation number G producing an individual i is defined as

$$s_{i,X}^G = \begin{cases} \frac{f_{pi} - f_{ci}}{f_{pi} - f_B}, & \text{if } f_{ci} \leq f_{pi} \\ 0, & \text{else} \end{cases} \quad (4)$$

where f_{pi} is the fitness of the best parent individual, f_{ci} is the fitness of the newly created child individual $\mathbf{i}$, and f_B is the fitness of the overall best individual included in the current population. Thus, the maximum success measure $s_{\max}^G = 1$ occurs, if a new overall best individual ($f_{ci} = f_B$) is found. Then, for each operator of type X (e.g. one-point crossover, uniform mutation, etc.) an average success value can be calculated

$$s_X^G = \frac{1}{N_X} \sum s_{i,X}^G \quad (5)$$

where N_X is the number of operator applications of type X in the respective generation G .

2.2.3. Operator diversity measure

The operator diversity measure analyzes the distribution of the individuals in the search space. The euclidean distance of an individual $\mathbf{i}$ originating from the application of an operator of type X in generation G to the current best individual $\mathbf{b}$ is calculated

$$d_{i,X}^G = \sqrt{\sum_{j=1}^n (i_j - b_j)^2} \quad (6)$$

where n is the number of genes of the genotype, and i_j and b_j are the components of the respective genotypes. A weighted euclidean distance could alternatively be used of the form

$$dw_{i,X}^G = \sqrt{\sum_{j=1}^n w_j (i_j - b_j)^2} \quad (7)$$

where w_j is a weighting factor for each component of the genotype. This weighting factor could serve the purpose of mapping the current gene values into intervals $[0, 1]$, but then the lower as well as the upper limit of the respective gene need to be known. Since the heterogeneous genotype concept does not generally require lower or upper limits for each gene, the gene values can theoretically grow to infinity. Thus, it is not always possible to define lower and upper limits for a gene and therefore the weighting factor could not be determined. As long as the genotype consists of identical genes, e.g. for typical numerical benchmark functions, such weighting factors have no influence on the optimization performance.

However, within this work no weighting factors are used and the diversity measure therefore analyzes at which absolute distance to the current best individual new individuals are created. The diversity measure for the application of an operator of type X in generation number G producing an individual $\mathbf{i}$ is defined as

$$t_{i,X}^G = \begin{cases} 0, & \text{if } f \leq f_B(1 + \varepsilon) \\ \frac{1}{1 + \frac{d_{i,X}^G}{d_{\max}^G} \cdot a}, & \text{if } f > f_B(1 + \varepsilon) \end{cases} \quad (8)$$

where $d_{\max}^G$ is the maximum euclidean distance between the current best individual and all individuals of the current population. The factor a is introduced for scaling purposes only and is set to $a = 9$. Individuals with fitness values close to the current best individual are rated with a diversity measure of 0, since these individuals do not essentially help to escape from this region. All other individuals are assigned with diversity measure values decreasing slightly with increasing euclidean distance to the current best individual. This way, operators exploring the region around the occupied region can be identified. Again, for each operator type X an average diversity value can be calculated for each generation G , i.e.

$$t_X^G = \frac{1}{N_X} \sum t_{i,X}^G \quad (9)$$

where N_X is the number of operator applications of type X .

2.2.4. Operator rate adaptation

The aim of the operator rate adaptation is to provide operator rates guaranteeing a balance between exploration and exploitation. Depending on the current bff value either the success or the diversity measure is applied. If the current bff value is below the target bff value l , the success ranking is employed in order to force the successful operators, see Fig. 5. On the other hand, the diversity measure is applied if the search stagnates and finds a lot of very similar individuals close to the current best solution.

An adaptation rate δ is introduced depending on the difference between the current bff value and the threshold value l .

$$\delta = \begin{cases} -\frac{\delta_{\max}}{l} (\text{bff} - l), & \text{if } \text{bff} \leq l \\ \frac{\delta_{\max}}{1-l} (\text{bff} - l), & \text{if } \text{bff} > l \end{cases} \quad (10)$$

By setting the target value l and the maximum adaptation value $\delta_{\max}$ the proportion between exploration and exploitation as well as the magnitude of adaptation can be controlled.

Fig. 6 illustrates the linear adaptation of the operator rates which always sum up to a total of 100%. On the left, the current rates of six operators are shown. Then, the operators are ranked according to the previously determined success or diversity measure. The operator fulfilling the respective measure the best is assigned with rank 1, the worst operator is assigned with rank 6. Subsequently, the operator rates are linearly increased or decreased for the next generation according to

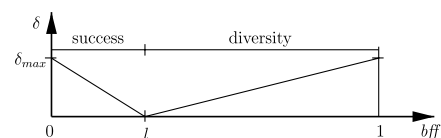


Fig. 5. Adaptation rate based on bff measure.

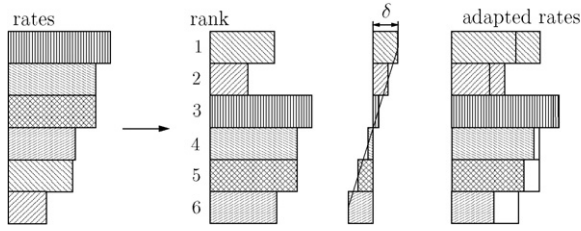


Fig. 6. Linear operator rate adaptation.

$$p_X^{G+1} = p_X^G + \left[\frac{N_O + 1 - 2R_X}{N_O - 1} \right] \delta \quad (11)$$

where G is the current generation, N_O is the total number of applied operators, and R_X is the rank of the operator of type X . Furthermore, a minimum operator rate $p_{\min}$ is guaranteed for each operator in order to prevent that single operators are completely excluded from the optimization process.

Practice has shown, that not only the measures of the current generation should be considered to rank the operators, but also some previous generations should be taken into account. If a lot of previous generations are considered, this may not reflect the operator performances of the current search state. On the other hand, only considering the operator performances of the current iteration may be statistically poorly founded. Therefore, the last five generations are averaged to success and diversity measures for all operators and the adaptation of the operator rates is performed each fifth generation.

3. Numerical examples

A series of benchmark functions is investigated to compare the performance of AORCEA with a non-adaptive EA. The general setup of the non-adaptive EA is exactly the same as it is for AORCEA in terms of population size, number of generations, selection, replacement, etc., with the only exception of constant operator rates. For the

Table 1

Test function setups defined by adaptation parameters

Setup	S_0	S_1	S_2	S_3	S_4	S_5	S_6	S_7
$\delta_{\max}$	0.0	0.05	0.05	0.05	0.1	0.1	0.2	0.2
l	–	0.1	0.25	0.5	0.1	0.25	0.1	0.25
ε	–	0.01	0.01	0.01	0.01	0.01	0.01	0.01

Setup S_0 is the reference optimization without adaptation.

investigations at hand four different genetic operators are chosen, namely one-point crossover (0.35), uniform crossover (0.35), uniform mutation (0.15), and gaussian mutation (0.15). The initial operator rates are given in parentheses which naturally remain constant for the non-adaptive optimization.

For each optimization problem the population size is $P = 2n$, where n is the problem dimensionality, and the maximum number of generations is set to $G_{\max} = 20n$ for unconstrained and $G_{\max} = 40n$ for constrained benchmark functions. The ε -value from Eq. (2) determining the bff value is set to 0.01 and the minimum operator rate $p_{\min}$ is set to 10% for each operator.

The sensitivity of AORCEA with respect to the parameter values of the maximum adaptation $\delta_{\max}$ and the target bff value l from Eq. (10) is investigated by using different combinations of these adaptation parameter values listed in Table 1. The setups S_1 to S_3 rather slowly adapt to the search state since the magnitude of adaptation $\delta_{\max}$ is relatively small. The setups S_4 to S_7 are characterized by increased $\delta_{\max}$ values and therefore adapt faster. Finally, the different l values should investigate which proportion of exploration and exploitation performs better.

A total of seven well-known unconstrained and four constrained functions listed in Tables 2 and 3, respectively are examined. The test functions F1–F3 are from De Jong's [1] test function suite. The further unconstrained functions are Ackley's function [23] (F4), a version of Griewank's function [24] (F5), the Michalewicz function [25] (F6), and the Rastrigin function from [26] (F7). The constrained functions C1 and C2 are taken from Floudas and Pardalos [27], C3 and C4 are from Hock and Schittkowski [28].

Table 2
Unconstrained benchmark functions

F	n	Function	Range of x_i
F1	10	$f(n, \mathbf{x}) = \sum_{i=1}^n x_i^2$	$x_i \in [-10.0, 10.0]$
F2	20	$f(n, \mathbf{x}) = \sum_{i=1}^{n-1} 100(x_i - x_{i-1}^2)^2 + (1 - x_{i-1})^2$	$x_i \in [-10.0, 10.0]$
F2a	50	$f(n, \mathbf{x}) = \sum_{i=1}^{n-1} 100(x_i - x_{i-1}^2)^2 + (1 - x_{i-1})^2$	$x_i \in [-10.0, 10.0]$
F3	5	$f(n, \mathbf{x}) = 30 + \sum_{i=1}^n [x_i]$	$x_i \in [-5.12, 5.12]$
F4	10	$f(n, \mathbf{x}) = -20e^{-0.2\sqrt{\frac{1}{n}\sum_{i=1}^n x_i^2}} - e^{\sum_{i=1}^n \cos(2\pi x_i)} + 20 + e$	$x_i \in [-32.768, 32.768]$
F5	10	$f(n, \mathbf{x}) = \frac{1}{4000} \sum_{i=1}^n (x_i - 100)^2 - \prod_{i=1}^n \cos\left(\frac{x_i - 100}{\sqrt{i+1}}\right) + 1$	$x_i \in [-600.0, 600.0]$
F6	10	$f(n, \mathbf{x}) = -\sum_{i=1}^n \left(\sin(x_i) \sin^{20}\left(\frac{x_i^2}{\pi}\right) \right)$	$x_i \in [0, \pi]$
F7	20	$f(n, \mathbf{x}) = 10n + \sum_{i=1}^n [x_i^2 - 10 \cos(2\pi x_i)]$	$x_i \in [-5.12, 5.12]$
F7a	50	$f(n, \mathbf{x}) = 10n + \sum_{i=1}^n [x_i^2 - 10 \cos(2\pi x_i)]$	$x_i \in [-5.12, 5.12]$

Table 3
Constrained benchmark functions

C	n	Function/constraints
C1	13	$f(\mathbf{x}, \mathbf{y}) = 5x_1 + 5x_2 + 5x_3 + 5x_4 - 5\sum_{i=1}^4 x_i^2 - \sum_{i=1}^9 y_i$ $2x_1 + 2x_2 + y_6 + y_7 \leq 10$ $2x_2 + 2x_3 + y_7 + y_8 \leq 10$ $-8x_2 + y_7 \leq 0$ $-2x_4 - y_1 + y_6 \leq 0$ $-2y_4 - y_5 + y_8 \leq 0$ $0 \leq y_i \leq 1, i = 1, 2, 3, 4, 5, 9$ $2x_1 + 2x_3 + y_6 + y_8 \leq 10$ $-8x_1 + y_6 \leq 0$ $-8x_3 + y_8 \leq 0$ $-2y_2 - y_3 + y_7 \leq 0$ $0 \leq x_i \leq 1, i = 1, 2, 3, 4$ $0 \leq y_i, i = 6, 7, 8$
C2	6	$f(\mathbf{x}, \mathbf{y}) = -10.5x_1 - 7.5x_2 - 3.5x_3 - 2.5x_4 - 1.5x_5 - 10y - 0.5\sum_{i=1}^5 x_i^2$ $6x_1 + 3x_2 + 3x_3 + 2x_4 + x_5 \leq 6.5$ $0 \leq x_i \leq 1$ $10x_1 + 10x_3 + y \leq 20$ $0 \leq y$
C3	7	$f(\mathbf{x}) = (x_1 - 10)^2 + 5(x_2 - 12)^2 + x_3^4 + 3(x_4 - 11)^2 + 10x_5^6 + 7x_6^2 + x_7^4 - 4x_6x_7 - 10x_6 - 8x_7$ $127 - 2x_1^4 - 3x_2^4 - x_3 - 4x_4^4 - 5x_5 \geq 0$ $196 - 23x_1 - x_2^2 - 6x_6^2 + 8x_7 \geq 0$ $-10.0 \leq x_i \leq 10.0, i = 1, \dots, 7$ $282 - 7x_1 - 3x_2 - 10x_3^2 - x_4 + x_5 \geq 0$ $-4x_1^2 - x_2^2 + 3x_1x_2 - 2x_3^2 - 5x_6 + 11x_7 \geq 0$
C4	10	$f(\mathbf{x}) = x_1^2 + x_2^2 + x_1x_2 - 14x_1 - 16x_2 + (x_3 - 10)^2 + 4(x_4 - 5)^2 + (x_5 - 3)^2 + 2(x_6 - 1)^2 + 5x_7^2 +$ $7(x_8 - 11)^2 + 2(x_9 - 10)^2 + (x_{10} - 7)^2 + 45$ $105 - 4x_1 - 5x_2 + 3x_7 - 9x_8 \geq 0$ $-10x_1 + 8x_2 + 17x_7 - 2x_8 \geq 0$ $8x_1 - 2x_2 - 5x_9 + 2x_{10} + 12 \geq 0$ $3x_1 - 6x_2 - 12(x_9 - 8)^2 + 7x_{10} \geq 0$ $-10.0 \geq x_i \geq 10.0, i = 1, \dots, 10$ $-3(x_1 - 2)^2 - 4(x_2 - 3)^2 - 2x_3^2 + 7x_4 + 120 \geq 0$ $-x_1^2 - 2(x_2 - 2)^2 + 2x_1x_2 - 14x_5 + 6x_6 \geq 0$ $-5x_1^2 - 8x_2 - (x_3 - 6)^2 + 2x_4 + 40 \geq 0$ $-0.5(x_1 - 8)^2 - 2(x_2 - 4)^2 - 3x_5^2 + x_6 + 30 \geq 0$

3.1. Results

For each test function and each setup a total number of 20 independent runs were performed to get statistically well-founded data. The numerous optimization runs produced a huge amount of data, too much to discuss them all in detail. Nevertheless, in order to compare the optimization performances of the different setups, Wilcoxon rank-sum tests [29] are used to determine whether the adaptive setups are significantly superior to the non-adaptive setup.

The Wilcoxon rank-sum test is a statistical significance test for assessing whether the difference in medians between two samples of observations is statistically significant, whereas the null hypothesis is that the two samples have a common median. Here, the first sample consists of the best fitness values of the 20 runs after the maximum number of generations of the non-adaptive setup. The second sample contains the best fitness values an adaptive setup, hence, each adaptive setup is compared to the non-adaptive setup by means of a rank-sum test. The p -value returns the significance for testing the null hypothesis. The alternative is that the median of one sample is shifted from the median of the other sample by a non-zero amount. Thus, for p -values below 5% a significant difference of the median values is very likely.

3.1.1. Unconstrained benchmark functions

The results presented in Tables 4 and 5 show the best and the median objective values after $20n$ generations as well as the Wilcoxon rank-sum test results for the optimization setups given in Table 1. Obviously, not all of the optimizations

are fully converged to their global optima, but nevertheless an interpretation of the optimization results is possible. The test function results are briefly discussed:

- F1: The median of the non-adaptive configuration (S_0) is clearly greater than the median values of all adaptive setups and only one best value of the adaptive setups (S_1) is slightly worse. Additionally, the p -value of the Wilcoxon rank-sum test is clearly below 5% for all adaptive setups indicating that they perform significantly better than the non-adaptive setup.
- F2: For both, the test functions F2 and F2a, the adaptive setups are again superior to the non-adaptive setup because the median as well as the best values are below the reference values of the setup S_0 . However, for function F2 the setup S_3 cannot be considered as significantly better and for function F2a the setups S_2, S_3, S_5 , and S_7 miss the 5% criterion.
- F3: The step function is difficult to be interpreted. Each setup finds the global optimum at least once, whereas the fast adapting setups (S_4 – S_7) more often solve the problem since even their median values coincide with the global optimum.
- F4: Similar to F1, the adapting setups show the better performance, i.e., their median values are clearly below the reference of setup S_0 . Furthermore, the p -values of the rank-sum test confirm this observation, they are all clearly below the 5% significance level. The convergence behavior of the different strategies are shown in Fig. 7a.
- F5: The median values as well as the p -values reveal that fast adaptation leads to a better convergence behavior

Table 4
Median and best values of the objective of the unconstrained benchmark functions after $20n$ generations

F	S_0		S_1		S_2		S_3	
	Median	Best	Median	Best	Median	Best	Median	Best
F1	0.14702	0.02521	0.065138	0.026211	0.086848	0.022536	0.058401	0.019852
F2	487.11	201.45	261.4	155.62	289.46	130.77	294.35	96.888
F2a	199.1	109.66	138.89	56.756	160.18	72.302	177.44	95.893
F3	1	0	1	0	1	0	1	0
F4	4.1241	1.7282	3.4377	1.6677	3.3178	2.7353	3.4691	2.4690
F5	0.23172	0.082854	0.15872	0.047653	0.15949	0.094708	0.17759	0.080719
F6	−7.7739	−8.8138	−8.1687	−9.1023	−8.2884	−9.1929	−8.2319	−9.4415
F7	78.192	54.029	67.613	45.946	70.062	42.211	75.997	35.735
F7a	287.13	212.66	282.96	244.39	276.18	202.39	291.65	206.17
F	S_4		S_5		S_6		S_7	
	Median	Best	Median	Best	Median	Best	Median	Best
F1	0.04524	0.014096	0.042572	0.005285	0.042878	0.016557	0.046939	0.016557
F2	305.29	122.83	254.15	104.97	261.1	156.78	221.04	105.07
F2a	125.21	87.379	152.44	51.56	146.52	56.837	151.28	45.583
F3	0	0	0	0	0	0	0	0
F4	3.1302	2.1135	3.0833	1.6939	3.1606	1.6684	2.7102	2.2463
F5	0.15339	0.057408	0.16099	0.058149	0.12516	0.042378	0.12604	0.050569
F6	−8.4073	−9.044	−8.2368	−8.7635	−8.3065	−8.7469	−7.9143	−9.1131
F7	64.886	50.821	69.566	54.50	75.37	43.425	71.325	50.322
F7a	279.63	215.26	281.12	239.38	283.68	223.46	290.39	253.01

The global optimum $F(\mathbf{x}^*)$ of all functions is 0.0 except for $F6(\mathbf{x}^*) = -9.66$.

Table 5
The p -values of the Wilcoxon rank-sum test results for the unconstrained benchmark functions

F	S_0	S_1	S_2	S_3	S_4	S_5	S_6	S_7
F1	1.0	4.493e ^{−4}	4.549e ^{−3}	4.045e ^{−3}	1.890e ^{−4}	1.890e ^{−4}	1.401e ^{−4}	2.931e ^{−4}
F2	1.0	3.656e ^{−2}	4.004e ^{−2}	1.004e ^{−1}	4.559e ^{−3}	2.509e ^{−2}	8.034e ^{−3}	4.045e ^{−3}
F2a	1.0	1.688e ^{−2}	5.222e ^{−2}	1.913e ^{−1}	1.524e ^{−2}	3.317e ^{−1}	4.549e ^{−3}	1.353e ^{−1}
F3	1.0	4.671e ^{−1}	5.491e ^{−1}	9.340e ^{−1}	3.398e ^{−1}	3.258e ^{−1}	4.248e ^{−2}	8.300e ^{−2}
F4	1.0	4.493e ^{−4}	1.713e ^{−3}	4.044e ^{−2}	1.890e ^{−4}	5.934e ^{−4}	1.204e ^{−4}	1.401e ^{−4}
F5	1.0	1.084e ^{−1}	2.509e ^{−2}	1.353e ^{−1}	1.688e ^{−2}	1.524e ^{−2}	3.185e ^{−3}	1.944e ^{−3}
F6	1.0	1.560e ^{−1}	6.735e ^{−2}	8.592e ^{−2}	2.762e ^{−2}	7.313e ^{−2}	2.509e ^{−2}	3.134e ^{−1}
F7	1.0	3.036e ^{−2}	1.084e ^{−1}	9.296e ^{−2}	2.063e ^{−2}	1.237e ^{−2}	1.084e ^{−1}	5.222e ^{−2}
F7a	1.0	9.702e ^{−1}	5.691e ^{−2}	2.471e ^{−1}	2.179e ^{−1}	9.108e ^{−1}	8.227e ^{−1}	9.404e ^{−1}

Values above the 5% significance level are printed in italic type.

for this benchmark function. Although all adaptive strategies show a lower median value than the non-adaptive one, the setups S_1 and S_3 slightly miss the 5% significance level. In Fig. 7b the convergence behavior of the different strategies are shown.

- F6: Again, all adaptive setups perform better than the reference setup S_0 in terms of the median values. However, only the setups S_4 and S_6 reach the significance level.
- F7: Although the median values of the adaptive setups are generally superior to the non-adaptive setup, the significance level is only rarely fulfilled. In particular the 50-dimensional benchmark function shows no significant differences in terms of the optimization performance.

3.1.2. Constrained benchmark functions

The results presented in Tables 6 and 7 analogously show the best and the median objective values after 40n generations for the constrained test functions. Due to the

fact that the fitness is a weighted sum of the objective and the numerous constraints it is necessary to define how the best individual is determined. In this case, it is not the individual with the lowest objective value, but it is the individual with the lowest overall fitness value. Therefore, the value for the best individual in Table 6 is the one with the lowest overall fitness which is not necessarily the one with the lowest objective value since always some constraints are active.

- C1: After 520 generations the search is converged for all strategies, hence, the median as well as the best values seem to be very close to each other. However, the p -value of the rank-sum test clearly reveals that the adaptive strategies perform significantly superior compared to the non-adaptive setup.
- C2: For this constrained test function the adaptive strategies work extremely efficient, since they all converge to a solution close to the global optimum. The best

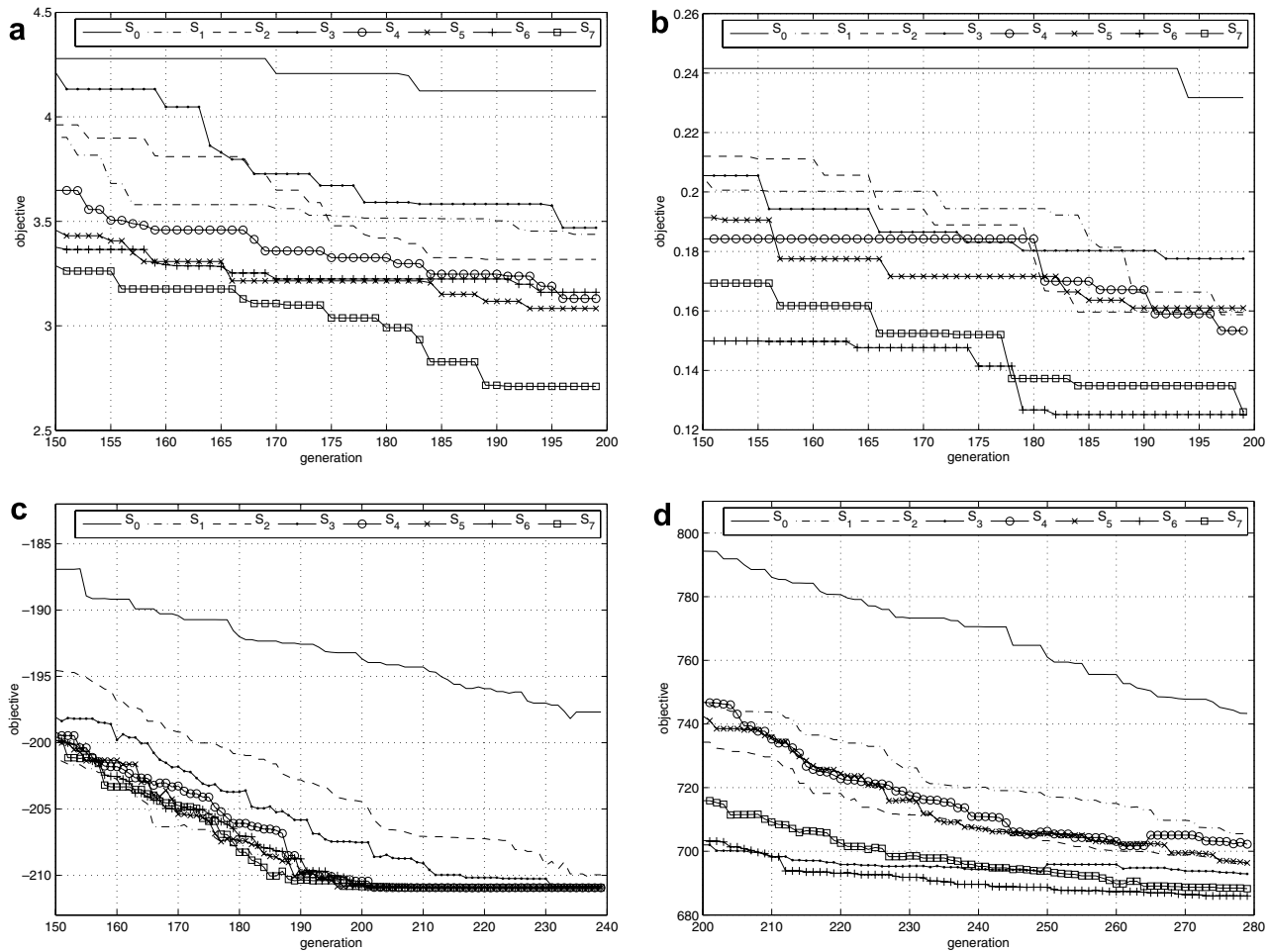


Fig. 7. Median values for different adaptation strategies compared to the non-adaptive strategy. (a) F4: Ackley's function, (b) F5: Griewank's function, (c) C2: Floudas's function and (d) C3: Hock's function.

Table 6

Median and best values of the objective of the constrained benchmark functions after $40n$ generations

F	S_0		S_1		S_2		S_3	
	Median	Best	Median	Best	Median	Best	Median	Best
C1	−13.922	−13.958	−13.944	−13.943	−13.95	−13.948	−13.939	−13.955
C2	−197.69	−210.93	−210.95	−212.69	−209.96	−212.16	−210.82	−212.74
C3	743.32	691.90	705.52	682.95	696.20	683.05	692.91	683.81
C4	39.939	32.803	33.406	28.638	34.449	27.552	35.266	29.350
F	S_4		S_5		S_6		S_7	
	Median	Best	Median	Best	Median	Best	Median	Best
C1	−13.955	−13.954	−13.952	−13.954	−13.942	−13.945	−13.948	−13.949
C2	−210.94	−212.20	−210.94	−212.92	−210.94	−212.21	−210.94	−212.25
C3	702.22	683.17	696.35	683.00	685.99	683.27	688.26	683.05
C4	33.956	27.319	32.515	27.654	32.431	27.322	34.743	29.307

The global optima are: $C1(\mathbf{x}^*) = -15.0$, $C2(\mathbf{x}^*) = -213.0$, $C3(\mathbf{x}^*) = 680.6300573$, and $C4(\mathbf{x}^*) = 24.3062091$.

solutions are all below -212.16 and the median values are smaller than -209.96 , see Fig. 7c. At the same point of the search process the non-adaptive strategy finds a best individual with an objective value of -210.93 and the median objective value is only -197.69 . Obviously,

also the p -values confirm the superiority of the adaptive strategies.

- C3: The convergence behavior depicted in Fig. 7d clearly shows, that the adaptive strategies again outperform the non-adaptive strategy. The adaptation of the operator

Table 7

The p -values of the Wilcoxon rank-sum test results for the constrained benchmark functions

F	S_0	S_1	S_2	S_3	S_4	S_5	S_6	S_7
C1	1.0	8.844e ⁻⁵	1.033e ⁻⁴	1.204e ⁻⁴	1.033e ⁻⁴	1.204e ⁻⁴	1.399e ⁻⁴	8.857e ⁻⁵
C2	1.0	1.964e ⁻⁴	4.045e ⁻³	1.237e ⁻²	4.045e ⁻³	3.981e ⁻⁴	1.823e ⁻³	1.087e ⁻³
C3	1.0	5.111e ⁻³	1.688e ⁻²	4.785e ⁻²	5.733e ⁻³	8.034e ⁻³	1.324e ⁻³	1.033e ⁻⁴
C4	1.0	1.204e ⁻⁴	6.424e ⁻³	1.560e ⁻¹	5.111e ⁻³	3.902e ⁻⁴	6.424e ⁻³	1.524e ⁻²

Values above the 5% significance level are printed in italic.

rates is obviously able to cope with optimization tasks searching along constraints.

- C4: Finally, the adaptive strategies are again better than the non-adaptive setup. The best solutions and the median objective value of each adaptive setup are below the reference values of S_0 and only the setup S_3 does not reach the 5% significance level.

Summarizing, for each constrained and unconstrained test function the adaptive strategies show superior median values. The only exception is setup S_7 of function F7a, where the median value of the non-adaptive setup is better. Furthermore, more than 50% of the adaptive setups of the unconstrained functions and almost 100% of the adaptive setups of the constrained functions perform significantly superior to the non-adaptive indicated by the Wilcoxon rank-sum test. Although there are some setups which are not significantly better, they still perform on at least the same level as the traditional EA. Unfortunately, it is impossible to recommend a parameter setup for $\delta_{\max}$ and l since none of the investigated setups clearly outperforms the other setups.

4. Tubular steel trellis frame

AORCEA is particularly developed for structural optimization tasks based on the heterogeneous genotype. Thus, an example for such a structural optimization is presented here by means of the tubular steel trellis frame of a motorcycle.

4.1. Parameterization

The symmetric Finite Element (FE) model depicted in Fig. 8 shows the original tube dimensions. A total of 15

tube dimensions defined by inner radius and thickness are subject to optimization, whereas identical tubes on each side of the frame are defined by the same parameter pair. The steering shaft tube dimensions and the positions of all tubes remain unchanged, hence, no functional or manufacturing constraints are violated. Two different parameterizations are examined:

- *Free tubes*: This parameterization allows individual inner radii and thicknesses for all 15 different tubes leading to a genotype consisting of 30 float-genes.
- *Custom tubes*: Only three custom tube types are permitted whose inner radii and thicknesses are defined by six float-genes. One of these three custom tubes is to be chosen for each of the tubes what is realized with 15 integer-genes assigning the first, second, or third custom tube.

4.2. Optimization formulation

After mapping the genotype to the FE-model depicted in Fig. 8b two load cases are simulated in order to estimate the quality of a given solution. The FE-model is composed of standard 2-node beam elements and incorporates the engine block which obviously influences the stiffness behavior of the entire system. The frame is clamped at the connection points to the swing arm and at an additional support at the headset, see Fig. 8b. First, the torsional stiffness is evaluated by applying a torsional moment M_T at the headset and solving a linear static analysis. A stiffness measure S is defined with the applied torsional moment and the resulting rotation λ_T between the swing arm axis and the headset axis:

$$S = \frac{M_T}{\lambda_T} \left[\frac{\text{Nm}}{^\circ} \right] \quad (12)$$

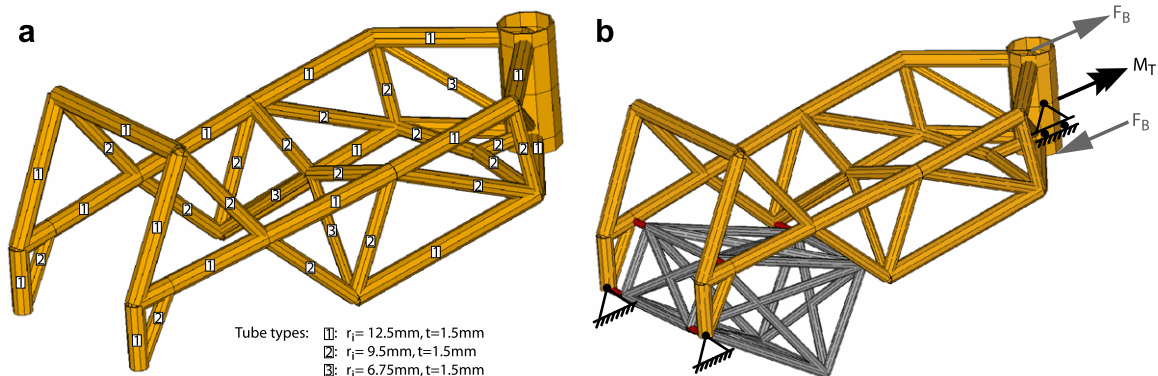


Fig. 8. FE-model of the tubular steel trellis frame of a motorcycle. (a) Original tube dimensions and (b) load case with simplified engine block.

The second load case simulates braking by applying a pair of forces F_B at the headset. After a second linear static analysis the maximum Von Mises stress $\sigma_{\max}$ is evaluated based on the nodal stress values.

The objective of this optimization is to reduce the total mass of the frame in compliance with the torsional stiffness and mechanical stress constraints.

$$\begin{aligned} \text{Minimize} \quad m &= \sum_{i=1}^E m_i \\ \text{subject to} \quad S &= 1330 \pm 5 \left[\frac{\text{N m}}{\circ} \right] \\ \sigma_{\max} &\leq 470.0 \text{ [MPa]}, \end{aligned} \quad (13)$$

where E is the number of finite elements and m_i the mass of element number i . The torsional stiffness is formulated as a target constraint which should be exactly met and the mechanical stresses must remain below the critical limit. The fitness function implementation is based on a weighted sum of ratings of the objective and both constraints (cf. Section 2.1).

The same seven adaptive setups are compared to the non-adaptive strategy defined in Table 1 by performing 20 runs for each setup. Two different operator configurations are investigated. First, the same four genetic operators are applied with the same starting values (one-point crossover (0.35), uniform crossover (0.35), uniform mutation (0.15), and gaussian mutation (0.15)) as they are used for the numerical examples. This configuration is denoted as free^α and custom^α , respectively, and the minimum operator rate $p_{\min}$ is analogously set to 10%. The second operator configuration (free^β and custom^β) includes some more genetic operators, i.e. one-point, two-point, uniform, inter-

mediate, segment, and hypercube crossover (0.1167), and uniform, deterministic uniform, gaussian, and deterministic gaussian mutation (0.075), whereas the minimum operator rate $p_{\min}$ is reduced to 5%. A brief definition of these genetic operator types is given in Section 2.1.

Furthermore, the population size for both, the free and the custom tube parameterization, is chosen to be 60 and the maximum number of generations is limited to 1000. In Tables 8 and 9 the results of both parameterizations are presented for all setups and the convergence plots are depicted in Fig. 9.

4.3. Results

For the operator configurations free^α and custom^α no significant differences between the convergence behavior of the adaptive and the non-adaptive setups can be seen during the first 100–200 generations. Presumably this behavior originates from the fact that the adaptive algorithm needs some generations in the beginning to identify efficient operators. Once these operators are detected their rates are increased and contribute to a superior convergence behavior. AORCEA seems to be quite insensitive to the adaptation parameters since all adaptive setups show approximately the same performances which are clearly superior to the non-adaptive setup. This observation is also confirmed by the p -values of the Wilcoxon rank-sum test listed in Table 9. All adaptive setups produce significantly lower median values indicated by p -values below the significance level of 5%. For the custom tubes parameterization the non-adaptive setup cannot find a best solution below 6.022 kg, whereas all adaptive strategies locate solutions

Table 8
Masses in kilograms of the best and median solutions of the tubular steel trellis frame optimization

Tubes	S_0		S_1		S_2		S_3	
	Median	Best	Median	Best	Median	Best	Median	Best
Free^α	5.9668	5.8095	5.8738	5.7921	5.8771	5.8086	5.8658	5.7836
Custom^α	6.0666	6.0222	5.9962	5.9315	5.9811	5.9518	5.9845	5.9373
Free^β	5.9390	5.8303	5.8835	5.7993	5.8953	5.8126	5.8657	5.8028
Custom^β	6.1001	6.0353	6.0274	5.9527	6.0095	5.9402	6.0214	5.9498
Tubes	S_4		S_5		S_6		S_7	
	Median	Best	Median	Best	Median	Best	Median	Best
Free^α	5.8898	5.8031	5.8597	5.7895	5.8644	5.8084	5.8690	5.8052
Custom^α	5.9746	5.9332	6.0152	5.9470	6.0222	5.9465	5.9922	5.9542
Free^β	5.8883	5.7991	5.8893	5.7919	5.8716	5.7836	5.8971	5.7918
Custom^β	6.0407	5.9497	6.0386	5.9583	6.0422	5.9493	6.0256	5.9389

Table 9
The p -values of the Wilcoxon rank-sum test results for the tubular steel trellis frame

Tubes	S_0	S_1	S_2	S_3	S_4	S_5	S_6	S_7
Free^α	1.0	1.507e ⁻³	2.535e ⁻⁴	8.918e ⁻⁴	1.018e ⁻³	8.918e ⁻⁴	7.795e ⁻⁴	5.934e ⁻⁴
Custom^α	1.0	1.628e ⁻⁴	8.857e ⁻⁵	1.401e ⁻⁴	5.167e ⁻⁴	1.204e ⁻⁴	3.384e ⁻⁴	5.934e ⁻⁴
Free^β	1.0	3.036e ⁻²	7.189e ⁻³	5.167e ⁻⁴	2.820e ⁻³	6.735e ⁻²	2.820e ⁻³	2.203e ⁻³
Custom^β	1.0	5.934e ⁻⁴	2.190e ⁻⁴	1.890e ⁻⁴	6.806e ⁻⁴	1.112e ⁻²	7.189e ⁻³	4.493e ⁻⁴

Values above the 5% significance level are printed in italic style.

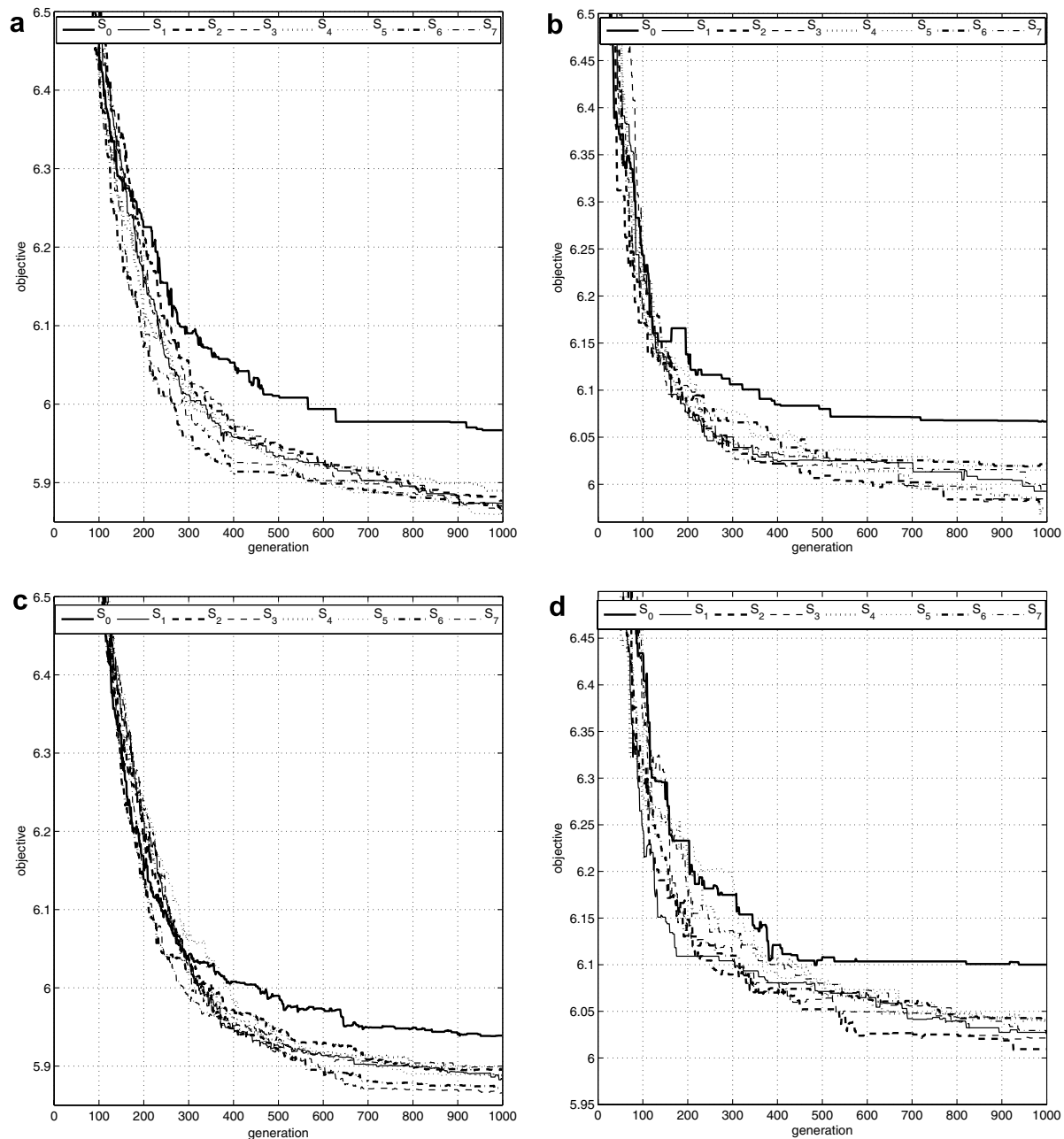


Fig. 9. Convergence behavior of different setups for the tubular steel truss frame optimization. The plot shows the median objective value of each setup. (a) Free^α tubes parameterization, (b) custom^α tube parameterization, (c) free^β tubes parameterization and (d) custom^β tube parameterization.

below 5.952 kg without violating the torsional stiffness or the mechanical stress constraint.

For the operator configurations free^β and custom^β the effect of adaptation occurs after approximately generation 400. Since there are much more operators involved and the minimum operator rates are relatively high, the adaptation cannot exert that much influence on the optimization performance. Nevertheless, all adaptive strategies are still superior compared to the non-adaptive strategy in terms of the best and the median fitness values. Additionally, all but one setup (S_5) are significantly better for this parameter configuration. All these results confirm the observations from the numerical examples in Section 3 where the

adaptive strategies always perform superior or at least equal to the non-adaptive algorithm.

The comparison between the configurations free^α and free^β reveals that the application of more operators helps the non-adaptive strategy to perform better. The converse behavior can be observed by comparison of the configurations custom^α and custom^β where the application of more genetic operators leads to generally worse optimization performances in terms of the median as well as the best solutions.

Surprisingly, the adaptation parameters seem to have a minor influence on the optimization performance. All adaptive optimization setups show a very similar

convergence behavior although their adaptation parameters are different. The convergence plots in Fig. 9 as well as the optimization performance measures in Table 9 confirm this observation. However, further investigations have shown that these adaptation parameters cannot be chosen arbitrarily. Other optimization runs with higher values for ε , $\delta_{\max}$, and l provide convergence behavior and best solutions which are extremely worse compared to the non-adaptive algorithm. Thus, a reasonable choice of the adaptation parameters in the range of the presented seven setups is definitely necessary.

5. Conclusion and outlook

AORCEA is tested on eleven well-known unconstrained and constrained numerical functions and compared to the performance of the non-adaptive strategy. For each of the test functions AORCEA performs at least slightly but most often significantly improved compared to the traditional algorithm. Additionally, AORCEA is applied to a structural optimization problem which reduces the mass of a steel trellis frame of a motorcycle while complying with the given stiffness and stress constraints. The adaptive strategy clearly outperforms the non-adaptive strategy in terms of convergence behavior and solution quality. Unfortunately, the results cannot identify an adaptation parameter setup which is clearly superior, but at least one can conclude that the performance of AORCEA is not extremely dependent on these parameters.

It is in the nature of EA development that there are always a lot of parameters decisively determining the optimization performance. Furthermore, these algorithms can only be tested on a very limited set of test functions or concrete problems from engineering practice. It is therefore necessary to develop robust algorithms which are very general in their nature to make them usable for a wide variety of optimization tasks. In the future, AORCEA will be used for structural optimization problems based on the heterogeneous genotype used in this work. Further steps in the development could be the incorporation of self-adaptive strategies for some particular genetic operators. As an example, the standard deviation determining the behavior of the gaussian mutation operator seems to have a crucial influence on the optimization performance in particular when it comes to the fine tuning towards the end of the optimization run. Some self-adaptive strategies are known but mostly they are developed for algorithms using the bit representation. It is a very promising approach to combine these self-adaptive approaches with AORCEA in order to increase the algorithms efficiency again.

Acknowledgement

This work was supported by the Swiss National Foundation (SNF-project 20002-103861).

References

- [1] De Jong K. The analysis of the behaviour of a class of genetic adaptive systems. PhD thesis, Department of Computer Science, University of Michigan, Ann Arbor, Michigan, 1975.
- [2] Grefenstette J. Optimization of control parameters for genetic algorithms. *Trans Syst Man Cybernet* 1986;16(1):122–8.
- [3] Schaffer JD, Caruana R, Eshelman I, Das R. A study of control parameters affecting online performance of genetic algorithms for function optimization. In: Schaffer JD, editor. *Proceedings of the 3rd international conference on genetic algorithms*. Morgan Kaufman; 1989. p. 51–60.
- [4] Angeline PJ. Adaptive and self-adaptive evolutionary computations. In: Palaniswami M, Attikiouzel Y, editors. *Computational intelligence: a dynamic systems perspective*. IEEE Press; 1995. p. 152–63.
- [5] Rechenberg I. *Evolutionsstrategie: Optimierung Technischer Systeme nach Prinzipien der Biologischen Evolution*, Frommann-Holzboog, Stuttgart-Bad Cannstatt, 1973.
- [6] Kakazu Y, Sakanashi H, Suzuki K. Adaptive search strategy for genetic algorithms with additional genetic algorithms. In: Männer R, Manderick B, editors. *Parallel problem solving from nature*, vol. 2. Amsterdam: North-Holland; 1992. p. 311–20.
- [7] Freisleben B, Härtfelder. Optimization of genetic algorithms by genetic algorithms. In: Albrecht R, Reeves C, Steele N, editors. *Artificial neural nets and genetic algorithms*. Springer-Verlag; 1993. p. 392–9.
- [8] Julstrom BA. What have you done for me lately? Adapting operator probabilities in a steady-state genetic algorithm. In: Eshelman LJ, editor. *Proceedings of the sixth international conference on genetic algorithms*. Morgan Kaufman; 1995. p. 81–7.
- [9] Tuson A, Ross P. Adapting operator settings in genetic algorithms. *Evolution Comput* 1998;6(2):161–84.
- [10] Davis L. Adapting operator probabilities in genetic algorithms. In: J.J. Grefenstette editor. *Proceedings of the 1st international conference on genetic algorithms and their applications*, Lawrence Erlbaum Associates, 1989. p. 61–9.
- [11] Smith RE, Smuda E. Adaptively resizing populations: algorithm, analysis and first results. *Complex Syst*. 1995;9(1):47–72.
- [12] Hinterding R, Michalewicz Z, Peachey TC. Self-adaptive genetic algorithm for numeric functions. In: Voigt H-M, Ebeling W, Rechenberg I, Schwefel H-P, editors. *Parallel problem solving from nature – PPSN IV*. Berlin: Springer; 1996. p. 420–9.
- [13] Bäck T. Self-adaption in genetic algorithms. In: Varela F, Bourguine P, editors. *Proceedings of the first European conference on artificial life*. Cambridge: MIT Press; 1992. p. 263–71.
- [14] Spears WM. Adapting crossover in evolutionary algorithms. In: McDonnell J, Reynolds R, Fogel D, editors. *Proceedings of the fourth annual conference on evolutionary programming*. MIT Press; 1995. p. 367–84.
- [15] Srinivas M, Patnaik L. Adaptive probabilities of crossover and mutation in genetic algorithms. *IEEE Trans Syst Man Cybernet* 1994;24(4):656–67.
- [16] Arabas J, Michalewicz Z, Mulawaka J. Gavaps – a genetic algorithm with varying population size. In: *Proceedings of the first IEEE international conference on evolutionary computing*. IEEE Press; 1994. p. 73–8.
- [17] Smith JE. Self adaptation in evolutionary algorithms. PhD thesis, Faculty of Computer Studies and Mathematics, University of the West of England, Bristol, 1998.
- [18] Smith J, Fogarty T. Adaptively parameterised evolutionary systems: self adaptive recombination and mutation in a genetic algorithm. In: *Proceedings of the fourth conference on parallel problem solving from nature*. Springer-Verlag; 1996. p. 441–50.
- [19] Eiben AE, Hinterding R, Michalewicz Z. Parameter control in evolutionary algorithms. *IEEE Trans Evolution Comput* 1999;3(2):124–41.

- [20] Goldberg D. Genetic algorithms in search optimization and machine learning. Reading, Massachusetts: Addison-Wesley Publishing Company; 1989.
- [21] König O. Evolutionary design optimization: tools and applications. PhD thesis, Swiss Federal Institute of Technology, Zürich, diss. ETH No. 15486, 2004.
- [22] Puisa R. The adaptive stochastic algorithms for structure optimisation of mechanical parts. PhD thesis, Vilnius Gediminas Technical University, Faculty of Mechanics, Department of Machine Building, UDK 531.8:51(043.3), 2005.
- [23] Ackley D. A connectionist machine for genetic hill climbing. Boston: Kluwer Academic Publishers; 1987.
- [24] Thomsen R, Krink T. Self-adaptive operator scheduling using the religion-based EA. In: Merelo JJ, editor. Parallel problem solving from nature – PPSN VII. Berlin: Springer; 2002. p. 214–23.
- [25] Michalewicz Z. Genetic algorithms + data structures=evolution programs. Springer; 1994.
- [26] Muhlenbein H, Schlierkamp-Voosen D. Predictive models for breeder genetic algorithms. *Evolution Comput* 1993;1(1):25–49.
- [27] Floudas CA, Pardalos PM. A collection of test problems for constrained global optimization algorithms. *Lecture Notes in Computer Science*, vol. 455. Springer-Verlag; 1987.
- [28] Hock W, Schittkowski K. Test examples for nonlinear programming codes. *Lecture notes in economics and mathematical systems*, vol. 187. Springer-Verlag; 1981.
- [29] Wilcoxon F. Individual comparisons by ranking methods. *Biomet Bull* 1945;1:80–3.